

# Current-State Analysis of nopCommerce

---

## 1. Overview

nopCommerce is a production-grade e-commerce platform built in **C# / ASP.NET Core**. Its baseline architecture is a **modular monolith** with a strict layered structure. It is capable software designed to operate as a standalone storefront — not as a commerce core in a wider operational ecosystem. That gap is the central tension of this analysis.

---

## 2. Layered Architecture

nopCommerce enforces a strict unidirectional dependency chain:

|                        |                                                           |
|------------------------|-----------------------------------------------------------|
| Nop.Web services)      | ← ASP.NET Core MVC (thin – delegates all logic to         |
| Nop.Web.Framework      | ← Filters, model binders, base controllers                |
| Nop.Services           | ← All business logic: orders, inventory, shipping, events |
| Nop.Core dependencies) | ← Domain entities, interfaces, events (no external        |
| Nop.Data PostgreSQL)   | ← Data access via LinqToDB (SQL Server, MySQL,            |

**Nop.Core** is the innermost layer — it owns the domain model (**Order**, **Product**, **Shipment**, **Warehouse**) and the event interfaces (**IEventPublisher**, **IConsumer<T>**). **Nop.Data** uses a generic **EntityRepository<TEntity>** backed by **TransactionScope** for atomic operations.

---

## 3. Plugin System

Plugins are discovered at startup and registered into ASP.NET Core DI via typed interfaces: **IShippingRateComputationMethod**, **IPaymentMethod**, **IWidgetPlugin**, and others. Any plugin can also react to domain events by implementing **IConsumer<TEvent>** — this is the mechanism used by third-party integrations like Brevo and Omnisend.

**Critical gap:** There is no fulfillment plugin interface. Shipping plugins only compute rates — they do not dispatch orders to warehouses or carriers. No **IFulfillmentProvider** or **IWarehouseConnector** exists.

---

## 4. Data Model — Orders, Inventory, Shipping

**Orders:** **Order** tracks status through three integer fields (**OrderStatusId**, **ShippingStatusId**, **PaymentStatusId**) mapped to enums. All status transitions are managed synchronously inside **OrderProcessingService**. Domain events are fired at key transitions: **OrderPlacedEvent**, **OrderPaidEvent**, **OrderStatusChangedEvent**.

**Inventory:** Stock quantity is stored in two mutually exclusive locations:

| Configuration                              | Storage                                                                  |
|--------------------------------------------|--------------------------------------------------------------------------|
| <code>UseMultipleWarehouses = false</code> | <code>Product.StockQuantity</code> (single integer)                      |
| <code>UseMultipleWarehouses = true</code>  | <code>ProductWarehouseInventory.StockQuantity + .ReservedQuantity</code> |

`ProductService.AdjustInventoryAsync()` performs a **direct database write** with no event published after the change — no `StockAdjustedEvent`, no notification to any external system.

**Warehouse entity:** Contains only `Name`, `AdminComment`, and `AddressId`. It is a logical grouping, not an integration point. It has no external system ID, no webhook, no sync mechanism.

**Shipping:** `Shipment` / `ShipmentItem` entities track fulfillment state. Domain events exist (`ShipmentCreatedEvent`, `ShipmentSentEvent`, `ShipmentDeliveredEvent`) but creating a shipment is a **manual admin action** — there is no automated dispatch to a carrier or WMS.

## 5. Event and Message System

`EventPublisher.PublishAsync<TEvent>()` resolves all `IConsumer<TEvent>` implementations from DI and calls `HandleEventAsync()` on each sequentially. 45 files across the codebase implement this interface, primarily in plugins.

**What it does not do:**

| Property                 | Reality                                                                                    |
|--------------------------|--------------------------------------------------------------------------------------------|
| <b>Durability</b>        | None — if the process crashes after the DB write, the event is lost                        |
| <b>Retry</b>             | None — a consumer that throws is logged and skipped                                        |
| <b>External delivery</b> | None — events never cross the process boundary; no bridge to RabbitMQ or any broker exists |

This is the most critical constraint for Scenario C. Any `IConsumer<OrderPlacedEvent>` that publishes to RabbitMQ works under normal conditions but has no delivery guarantee if the broker is unavailable. Without an outbox pattern, reliable external integration is not achievable.

## 6. Deployment Model

- **Single process:** web, services, plugins, and background tasks all run together
- **Single database:** all data (orders, stock, customers, shipments, warehouses) in one schema
- **Background tasks:** `ScheduleTask` runs as in-process polling loops, not a separate worker
- **Docker Compose:** available but only nopCommerce + database — no broker, no external services

## 7. Pressure Points Against Omnichannel Evolution

**P1 — Event bus has no durability.** `IEventPublisher` is in-process and fire-and-forget. Reliable WMS integration requires an outbox pattern.

**P2 — Stock is owned and isolated in nopCommerce's DB.** An external WMS has no channel to push stock updates in. `AdjustInventoryAsync` publishes no events, making reactive sync impossible.

**P3 — No fulfillment abstraction.** After order placement, the path to physical fulfillment is entirely manual. There is no hook to dispatch an order to a warehouse.

**P4 — Inventory deduction is synchronous at order placement.** There is no reservation/compensation model. If a warehouse rejects an order later, nopCommerce has no native compensating path.

**P5 — Warehouse concept is internal and non-integrable.** The `Warehouse` entity cannot represent an external WMS location without building a full mapping layer from scratch.

---

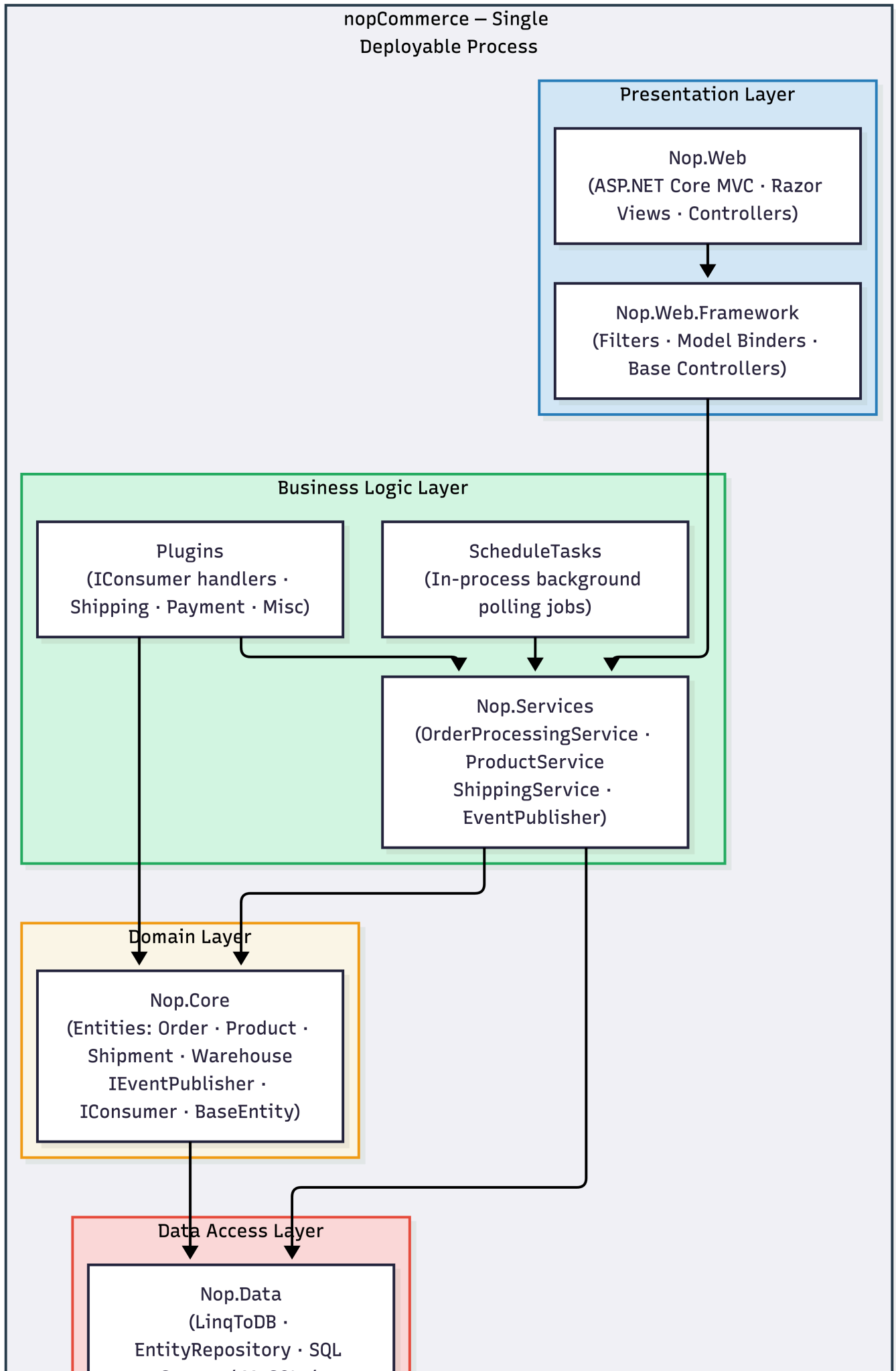
## 8. Architectural Seams — Where Integration Is Possible

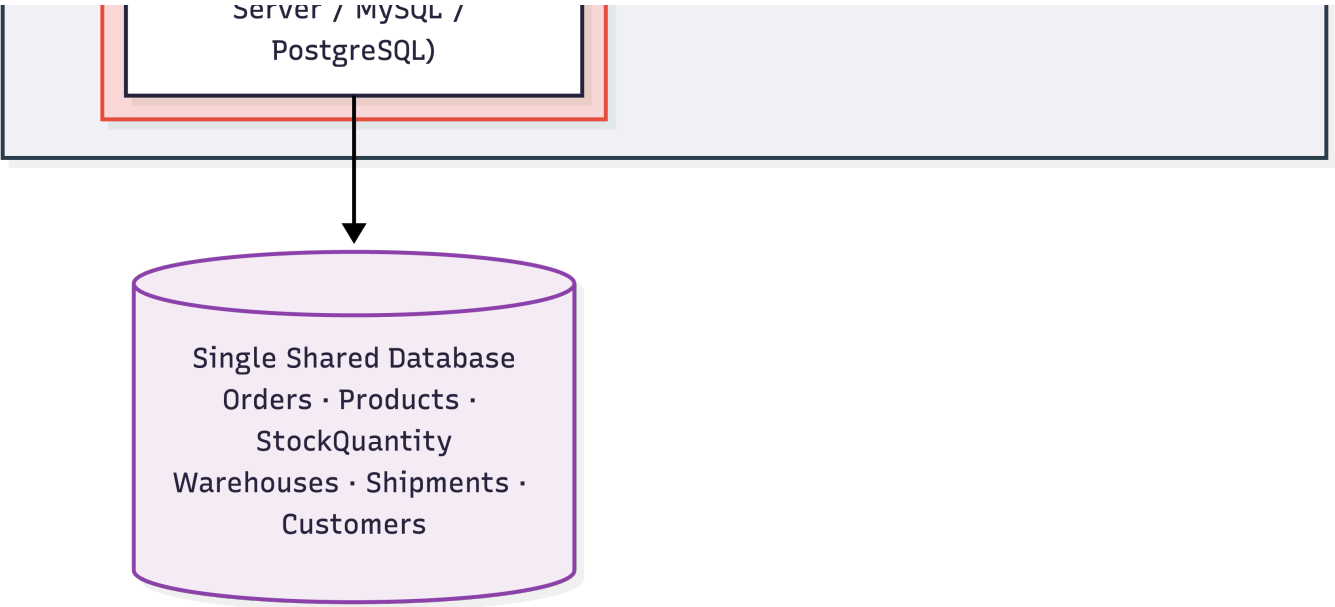
| Seam                  | Mechanism                                                 | Reliability concern         |
|-----------------------|-----------------------------------------------------------|-----------------------------|
| Order events          | <code>IConsumer&lt;OrderPlacedEvent&gt;</code> plugin     | Needs outbox for durability |
| Shipment events       | <code>IConsumer&lt;ShipmentCreatedEvent&gt;</code> plugin | Same                        |
| Inbound stock updates | Plugin HTTP endpoint                                      | Needs auth + idempotency    |
| Polling sync          | <code>IScheduleTask</code> implementation                 | Stale state window          |

The most viable path: a plugin that consumes `OrderPlacedEvent` and writes to an **outbox table** in the same transaction, drained by a separate worker that publishes to RabbitMQ.

---

## 9. As-Is Architecture Diagram





10. Summary

| Area             | Finding                               | Implication for Scenario C                    |
|------------------|---------------------------------------|-----------------------------------------------|
| Architecture     | Modular monolith, strict layers       | Safe to extend; risky to split without seams  |
| Event system     | In-process, no durability, no retry   | Outbox pattern required                       |
| Inventory        | DB-owned integer, no events on change | New integration surface needed                |
| Warehouse entity | Name + address only                   | Full mapping layer must be built              |
| Fulfillment      | No dispatch abstraction               | Order-to-warehouse flow designed from scratch |
| Deployment       | Single process + single DB            | Outbox worker must be a separate deployable   |